Apigee X Simulator
Interview-Prep Scenarios & Expected Request/Response

16 additional scenarios - different from the 18 in the first Local Test Guide - built from your actual project's source (seed.ts, policyCatalog.ts, policyExecutors.ts, flowTypes.ts). Each one includes exact steps, the exact request/response you should see, and an **Interview angle** note on the Apigee concept it demonstrates, so you can explain the "why", not just click through the UI.

Login: `admin@apigee-sim.local` / `Apigee123!`. All steps use the Trace page unless noted otherwise.

Security & Identity

1. VerifyAPIKey via Query Parameter (not header)

Proxy: CustomerService-v1 (VA-VerifyKey uses the policy's DEFAULT config - apiKeyLocation=query, apiKeyParam=apikey - unlike ProductCatalog-v1 which overrides to a header).

Steps:

1. Apps page: copy any app's Consumer Key.
2. Trace: Method GET, Path /customers?apikey=, no headers, Run Trace.
3. Then retry with the key removed from the URL.

Request:

```
GET /customers?apikey=6f2a...redacted...
(no Authorization/x-api-key header needed)
```

Expected response / result:

```
VA-VerifyKey: success
variables: verifyapikey.client_id, verifyapikey.developer.app.name
Without the query param: 401, "Failed to resolve API Key variable request.apikey"
```

Interview angle: Real Apigee's VerifyAPIKey policy resolves the key from a that can point at request.queryparam.* OR request.header.* - interviewers often ask you to explain both placements and when you'd pick one (query param = simpler for quick testing/curl; header = the more "correct" production convention).

2. Developer App revocation breaks both VerifyAPIKey and OAuthV2

Apps page.

Steps:

1. Open any Developer App, set its status to "revoked" (or delete it).
2. Re-run scenario 1's successful call with the same key.
3. Also retry the OAuthV2 GenerateAccessToken call (Local Test Guide, test 3) with that app's client_id/secret.

Request:

```
GET /customers?apikey=<same key as before>
```

Expected response / result:

```
401 "Invalid API Key" (VerifyAPIKey re-checks app.status === "approved" on every request)
OAuthV2 GenerateAccessToken: 401 "Client authentication failed"
```

Interview angle: This is the credential-lifecycle question interviewers like: revoking a Developer App must immediately invalidate BOTH API-key and OAuth access for every product tied to it - prove you understand that the check happens per-request, not just at token-issue time.

3. AccessControl actually blocking traffic (via the PreProxyFlowHook shared flow)

Global PreProxyFlowHook -> SharedFlow "Common-Security-Flow" -> SF-AccessControl, applies to EVERY proxy in "prod" only.

Steps:

1. Shared Flows page: open Common-Security-Flow, edit SF-AccessControl's config: ipList = 203.0.113.1, action = DENY (this matches Trace's default simulated client IP).
2. Trace: any proxy (e.g. CustomerService-v1), environment prod, with a valid API key.
3. Revert ipList back to 198.51.100.0/24 afterwards.

Request:

```
GET /customers?apikey=<valid key>  (environment: prod)
```

Expected response / result:

```
Timeline: "PreProxyFlowHook -> SharedFlow:Common-Security-Flow" phase, SF-AccessControl = error, 403
Every later step, including VA-VerifyKey, shows "skipped due to earlier fault"
```

Interview angle: Flow Hooks + Shared Flows let you enforce one security policy across an entire environment without touching every individual proxy - a core "reusability" talking point in Apigee interviews. Also demonstrates that a fault in PreProxyFlowHook short-circuits the ENTIRE remaining flow.

Mediation & Transformation

4. AssignMessage - overriding the response payload, not just headers

Any proxy, e.g. CustomerService-v1 postResponse.

Steps:

1. Open AM-AddHeader (or add a new AssignMessage), Assign To = response.
2. Set Payload to: {"overridden": true, "note": "served by simulator"}
3. Trace GET /customers with a valid key.

Request:

```
GET /customers?apikey=<valid key>
```

Expected response / result:

```
Response body is now exactly {"overridden": true, "note": "served by simulator"} - the real target's JSON is replaced entire
AM-AddHeader: success
```

Interview angle: Shows AssignMessage can fully replace a payload, not just add headers - useful when explaining response mocking/canary responses without touching the real backend.

5. ExtractVariables pulling a field out of the target's real response

Proxy: OrderService-v1, attach ExtractVariables to postResponse.

Steps:

1. Add ExtractVariables: Source=response, JSONPath=$[0].id, Variable Name=order.first.id, attach to PostFlow (response).
2. Trace GET /orders with a valid JWT (use the GOOD token from the Local Test Guide Appendix A).

Request:

```
GET /orders  Header: Authorization: Bearer <GOOD JWT>
```

Expected response / result:

```
ExtractVariables: success
Variables Created panel shows order.first.id = 1 (the real jsonplaceholder.typicode.com/posts[0].id)
```

Interview angle: Demonstrates pulling data out of a live target response into a flow variable for later use (e.g. logging, conditional routing, or a follow-up ServiceCallout) - a very common "how do you pass data between policies" interview question.

6. ServiceCallout - an out-of-band enrichment call

Any proxy, e.g. WeatherProxy-v1 PreFlow.

Steps:

1. Add ServiceCallout: Target URL=https://jsonplaceholder.typicode.com/todos/1, Method=GET, Response Variable=calloutResponse.
2. Trace GET /weather.

Request:

```
GET /weather
```

Expected response / result:

```
ServiceCallout: success, message mentions "returned 200"
Variables Created shows calloutResponse = {"userId":1,"id":1,"title":"...","completed":false}
The MAIN response is still the weather data - the callout result is separate
```

Interview angle: Key distinction to articulate: ServiceCallout makes a side call that does NOT replace the main target response - it's for enrichment/validation mid-flow, unlike changing the TargetEndpoint itself.

7. RaiseFault short-circuits everything after it

Any proxy, e.g. ProductCatalog-v1 PreFlow, placed BEFORE VA-VerifyKey.

Steps:

1. Add RaiseFault as the FIRST PreFlow policy: statusCode=403, reasonPhrase=Forbidden, errorMessage={"error":"maintenance window"}
2. Trace GET /products with a normally-valid API key.

Request:

```
GET /products?x-api-key=<valid key>  (header)
```

Expected response / result:

```
Timeline: RaiseFault = error, 403
VA-VerifyKey, Quota-Limit, RC-ProductCache: all show "skipped due to earlier fault"
Response body: {"error": "maintenance window"}
```

Interview angle: Great way to demonstrate you understand fault propagation order - once ANY step in a flow raises a fault, every remaining step in that phase is skipped, and ProxyEndpoint PostFlow still runs afterward (mirrors real Apigee's DefaultFaultRule).

8. JSON to XML and back, request vs response direction

Any proxy, e.g. CustomerService-v1.

Steps:

1. Add JSONToXML: Source=response, attach to PostFlow.
2. Trace GET /customers with a valid key.
3. Then swap it for XMLToJSON with Source=request on a POST call with a JSON body, to see the mirror direction.

Request:

```
GET /customers?apikey=<valid key>
```

Expected response / result:

```
Response Content-Type becomes application/xml, body is now <root>...</root> wrapped XML
JSONToXML: success
```

Interview angle: Classic legacy-integration question: converting between XML-only backends and JSON-only clients (or vice versa) at the edge, and knowing which side (request/response) each direction applies to.

Routing, Quota Layers & Lifecycle

9. Route Rule matching on path suffix, not just verb

Proxy: MultiRouteDemo-v1 Develop tab.

Steps:

1. Add Route Rule: Base Path Suffix=/items/urgent, Target Endpoint=write-target (leave Verb blank so it matches all methods), place it ABOVE the "reads" rule.
2. Trace GET /items/urgent, then GET /items (no suffix).

Request:

```
GET /items/urgent
GET /items
```

Expected response / result:

```
/items/urgent: Routing step -> "write-target" regardless of GET verb (path match wins because this rule is evaluated first)
/items (plain): Routing step -> "read-target" (falls through to the verb-based rule)
```

Interview angle: Shows you understand Route Rules are evaluated top-to-bottom and can combine verb AND path conditions - order matters, exactly like real Apigee's RouteRule/Condition evaluation.

10. Product-level quota vs a proxy's own Quota policy

ProductCatalog-v1 has its own Quota-Limit policy (5000/day). It's also included in the Premium-Tier API Product, which separately declares quotaLimit=100000/month.

Steps:

1. Open Products > Premium-Tier, note quotaLimit/quotaInterval/quotaTimeUnit fields at the PRODUCT level.
2. Open ProductCatalog-v1's Quota-Limit policy, note its independent config.

Request:

```
(no Trace call needed for this one - it's a configuration comparison)
```

Expected response / result:

```
Two independent quota layers exist: the API Product's quota (meant to gate a developer app's overall usage across every prox
```

Interview angle: A frequently-asked distinction: API Product quotas are entitlement/business-tier limits tied to a developer app; a Quota POLICY inside a proxy is enforced in the flow itself. Both can be present, and the proxy-level one applies even to calls that bypass product entitlement checks entirely.

11. Revision cloning and independent per-environment deployment

Any proxy, e.g. WeatherProxy-v1, Overview tab.

Steps:

1. Click "+ Create new revision" (creates revision 2, a copy of revision 1's policies/flows).
2. Edit revision 2 only (e.g. change SpikeArrest rate to 5ps).
3. Deploy revision 2 to dev; leave prod on revision 1.
4. Trace against dev (uses rev 2's 5ps) vs prod (uses rev 1's 20ps).

Request:

```
GET /weather  (environment: dev vs prod)
```

Expected response / result:

```
dev: SpikeArrest blocks after ~5 rapid calls
prod: SpikeArrest tolerates ~20 rapid calls - proving the two environments are genuinely running different revisions
```

Interview angle: Revision management is core Apigee lifecycle knowledge: you edit a new revision without disturbing what's live, test it in a lower environment, then promote/deploy it to prod only when ready - never edit a deployed revision in place in real Apigee.

12. OpenAPI import auto-generates conditional flows per path+verb

Any proxy, Overview tab actions -> Import OpenAPI.

Steps:

1. Paste a small spec, e.g.: {"openapi":"3.0.0","paths":{"/items":{"get":{},"post":{}},"/items/{id}":{"get":{},"delete":{}}}}
2. Click Import.
3. Open the proxy's Develop tab, check Conditional Flows.

Request:

```
(Import OpenAPI dialog, not a Trace call)
```

Expected response / result:

```
4 conditional flows appear, one per path+verb pair: GET /items, POST /items, GET /items/{id}, DELETE /items/{id}
Each has condition.verb and condition.basePathSuffix pre-filled from the spec
```

Interview angle: Spec-driven proxy generation is a real Apigee feature (and a differentiator vs hand-building every conditional flow) - good to mention you understand OpenAPI can bootstrap the routing skeleton, which you then attach policies to.

13. Undeploying one environment doesn't affect the others

Any multi-env proxy, e.g. CustomerService-v1, Deploy tab.

Steps:

1. Confirm it's deployed to dev, test, and prod.
2. Click Undeploy on dev only.
3. Trace against dev (should now fail with 409/not deployed) vs test (still works).

Request:

```
GET /customers?apikey=<valid key>  (environment: dev, then test)
```

Expected response / result:

```
dev: error - "Proxy \"CustomerService-v1\" is not deployed to \"dev\"" (HTTP 409)
test: unaffected, normal 200 response
```

Interview angle: Deployments are per-(proxy revision, environment) records - undeploying one is fully isolated from the others. This is the kind of "what happens if..." operational question interviewers ask to check you understand the deployment model, not just the policy model.

14. KVM - organization-wide vs environment-scoped values (admin feature, not a runtime policy)

KVM page.

Steps:

1. Open the "global-config" KVM (scope=organization) - note entries support.email and brand.name are visible everywhere.
2. Open the "dev-secrets" KVM (scope=environment, dev only, encrypted=true) - note api.upstream.secret only exists for dev.

Request:

```
(no Trace call - this is a data/config feature; there's no KeyValueMapOperations runtime policy in this build)
```

Expected response / result:

```
Org-scoped KVM entries are visible/usable from any environment; environment-scoped entries (like dev-secrets) are isolated t
The "encrypted" flag models Apigee's encrypted KVM entries (write-only in the real product - values aren't shown back once s
```

Interview angle: Be ready to explain KVM scope levels (organization / environment / (real Apigee also has API-proxy-scoped)) and why you'd choose encrypted KVMs for secrets instead of hardcoding them in a policy's XML.

Observability

15. Reading the Analytics dashboard after generating traffic

Analytics page (dashboard already has ~30 days of seeded historical data for CustomerService-v1, OrderService-v1, ProductCatalog-v1, WeatherProxy-v1).

Steps:

1. Open Analytics, filter by proxy = OrderService-v1, environment = prod.
2. Note the latency and error-rate charts, then run a few real Trace calls against OrderService-v1 to compare live behavior against the historical trend.

Request:

```
(Analytics page - historical mock data, no live Trace call required for this step)
```

Expected response / result:

```
Charts show requests/latency/error-rate over the last 30 days, with OrderService-v1's baseline ~180ms latency and ~5% error
```

Interview angle: Shows you can navigate from "a single request's trace" (micro view) to "aggregate API health over time" (macro view) - interviewers like candidates who connect Trace-level debugging to Analytics-level SLA/health monitoring.

16. Monitoring Alerts - thresholds vs actual values

Monitoring page.

Steps:

1. Open Monitoring, review any seeded alerts (severity, metric, value vs threshold).
2. Compare a "critical" alert's metric (e.g. error_rate) against what Analytics shows for that same proxy/environment.

Request:

```
(Monitoring page, no Trace call required)
```

Expected response / result:

```
An alert's value/threshold fields let you explain, concretely, why it fired (e.g. error_rate value 0.08 vs threshold 0.05)
```

Interview angle: Ties back to a very common ops interview question: "how would you know a proxy is unhealthy before customers complain?" - alerting on metric thresholds, not just eyeballing dashboards.